
UNIFIED
MODELING
LANGUAGE
TM

Historia de UML



Ivar Jacobson (Objectory)

Harel (state charts)

Odell

Meyer

Grady Booch (Booch)

James Rumbaugh (OMT)

UNIFIED
MODELING
LANGUAGE

Diagramas de estructura

- Diagrama de Clases
- Diagrama de Paquetes
- Diagrama de Componentes
- Diagrama de Objetos
- Diagrama de Despliegue

Diagramas de Comportamiento

- Diagrama de Casos de Uso
- Diagramas de Interacción
  - Diagrama de Secuencia
  - Diagrama de Colaboración
- Diagrama de Máquinas de estado
- Diagrama de Actividades

Diagramas de estructura

- **Diagrama de Clases**
- Diagrama de Paquetes
- Diagrama de Componentes
- Diagrama de Objetos
- Diagrama de Despliegue

Diagramas de Comportamiento

- Diagrama de Casos de Uso
- Diagramas de Interacción
  - **Diagrama de Secuencia**
  - Diagrama de Colaboración
- Diagrama de Máquinas de estado
- Diagrama de Actividades

Diagrama de Clases

Una clase es una descripción de conjunto de objetos que comparten los mismos atributos, operaciones, relaciones y semántica.

Una clase es representada gráficamente por cajas con tres compartimientos

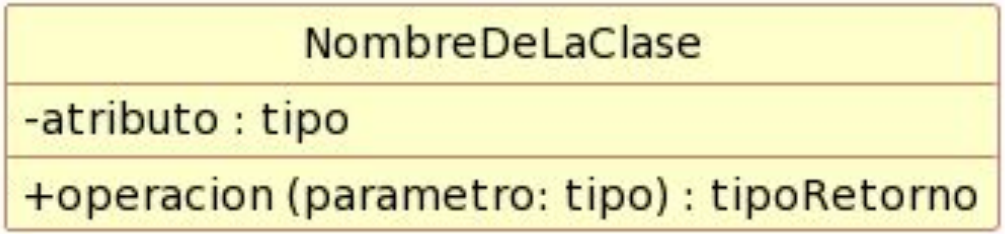
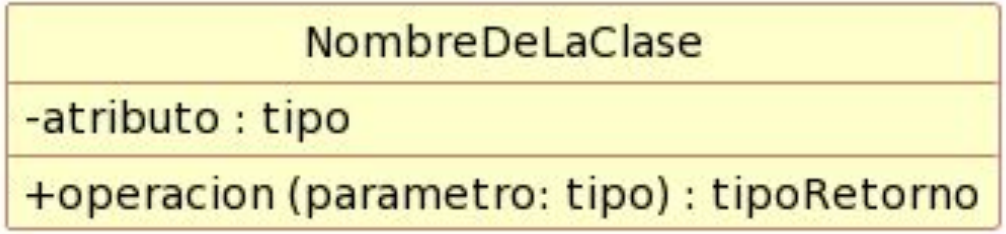
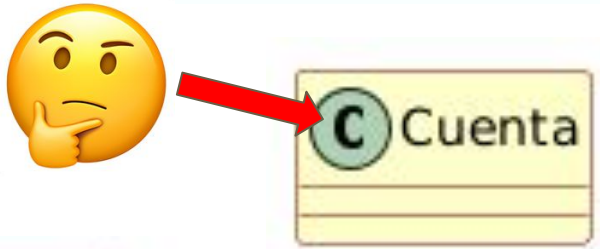


Diagrama de Clases



- Nombre de la clase
  - singular
  - debe comenzar con Mayúscula
  - estilo CamelCase



- Si la clase es abstracta
  - cursiva / Itálica
  - estereotipo <<abstract>>

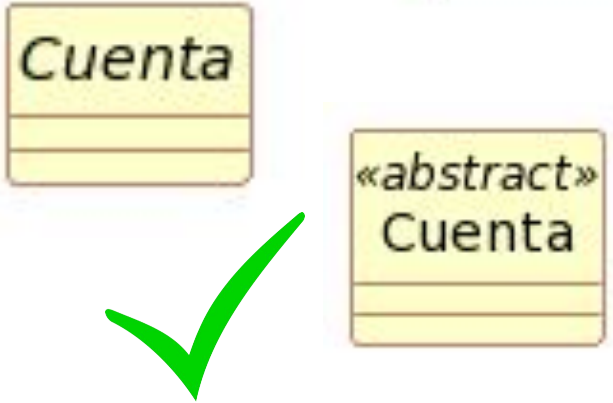
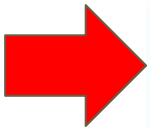


Diagrama de Clases - Atributos

visibilidad nombre: tipo = valor por defecto

- visibilidad
  - privada (-)
  - protegida (#)
  - pública (+)
  - paquete (~)
- nombre
  - estilo camelCase
  - comienza con minúscula
- tipo
  - tipos UML: Integer, Real, Boolean, String
- valor por defecto



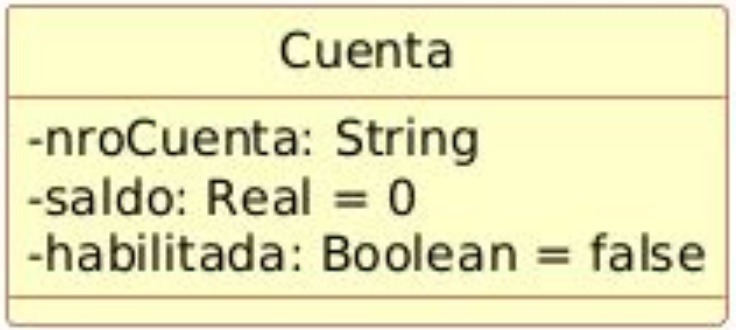
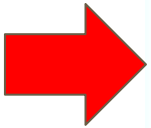


Diagrama de Clases - Operaciones

visibilidad nombre (parámetro: tipo): tipo retorno

- visibilidad
  - privada (-)
  - protegida (#)
  - pública (+)
  - paquete (~)
- nombre
  - estilo camelCase
  - comienza con minúscula
- Parámetros
  - nombre: estilo camelCase
- tipo de retorno
  - Si no retorna nada, no se especifica
  - SI retorna un objeto, se indica de qué clase
  - Si retorna una colección, se indica el nombre de la clase [*]



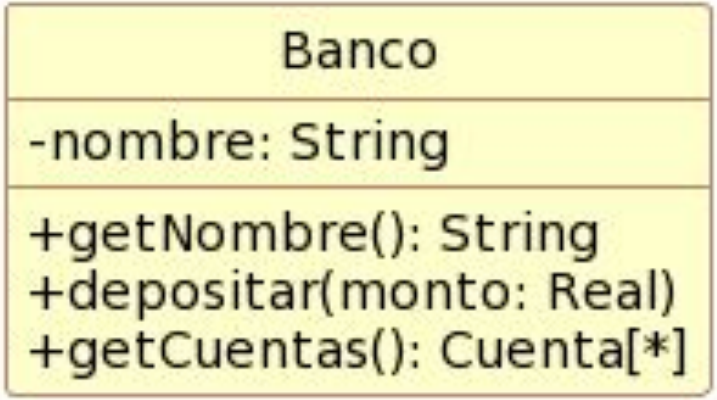
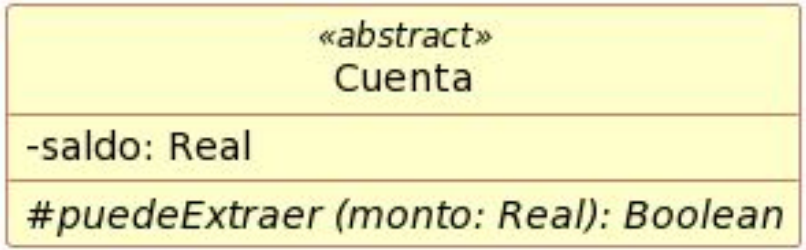


Diagrama de Clases - Operaciones

visibilidad nombre (parámetro: tipo): tipo retorno

-
- si el método es **abstracto**
  - *cursiva*
  - con estereotipo <<abstract>>
- si el método es un **constructor**
  - con estereotipo <<create>>



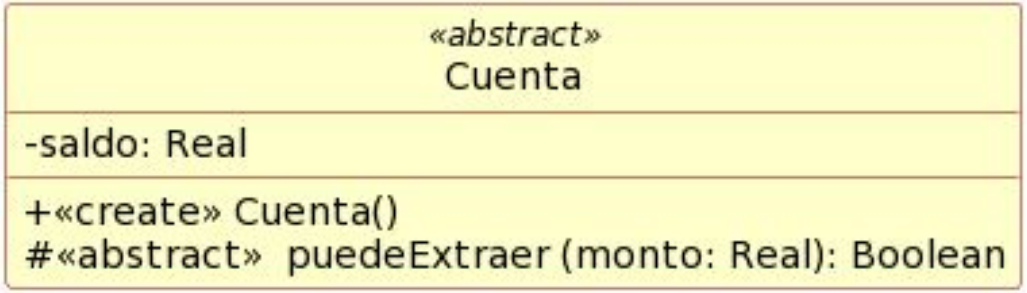


Diagrama de Clases - Relaciones

Asociaciones 

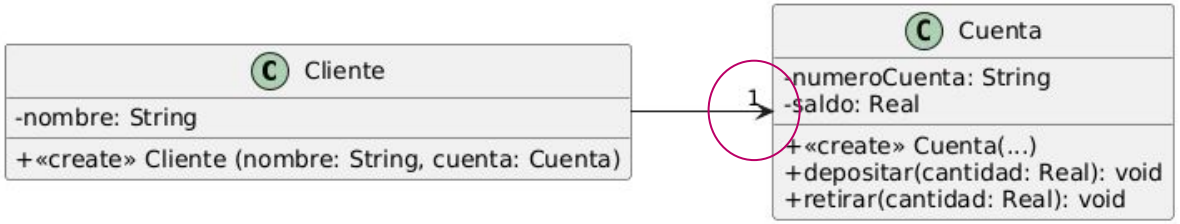
Herencia (Generalización) 

Implementación 

Dependencia 

Diagrama de Clases - Asociación

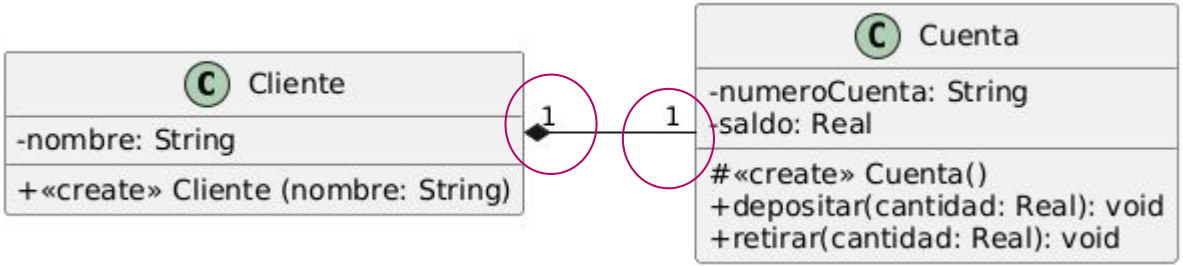
- **navegabilidad**
- multiplicidad
- nombre de rol
- tipo:
  - simple
  - agregación
  - composición




```
public class Cliente {
        private Sring nombre;
        private Cuenta cuenta;
}
public class Cuenta {
        private String numeroCuenta;
        private double saldo;
}
```

Diagrama de Clases - Asociación

- **navegabilidad**
- multiplicidad
- nombre de rol
- tipo:
  - simple
  - agregación
  - composición




```
public class Cliente {
        private String nombre;
        private Cuenta cuenta;
 }

  public class Cuenta {
        private String numeroCuenta;
        private double saldo;
        private Cliente cliente;
}
```

Diagrama de Clases - Asociación

- navegabilidad
- **multiplicidad**
- nombre de rol
- tipo:
  - simple
  - agregación
  - composición

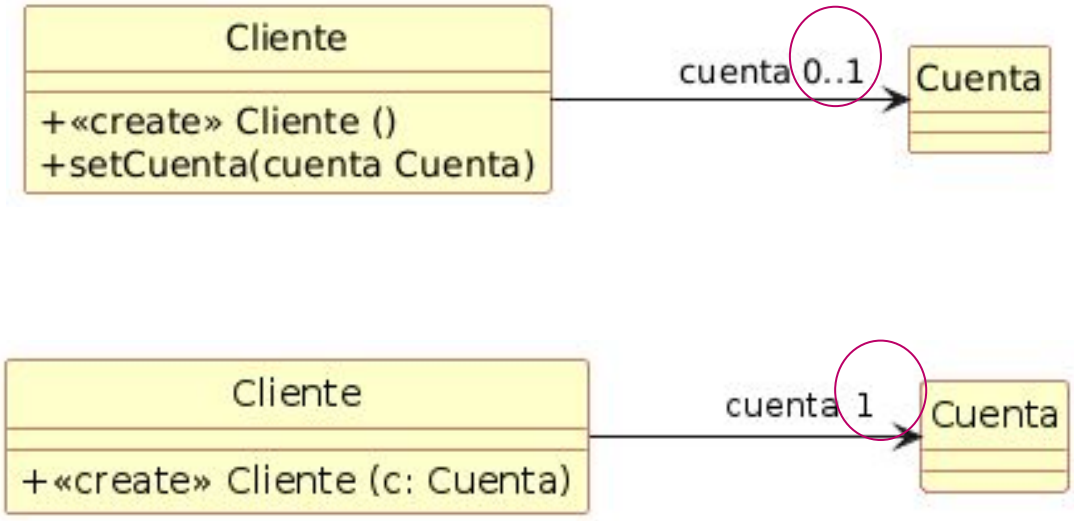
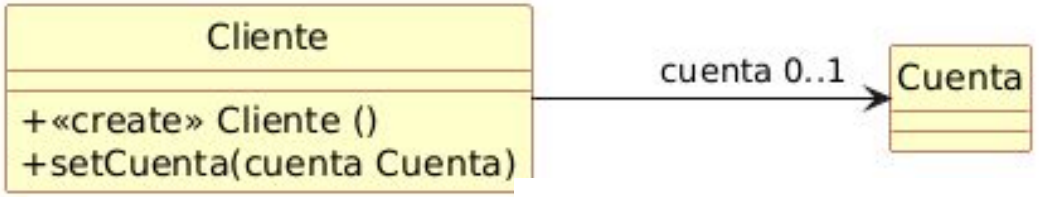
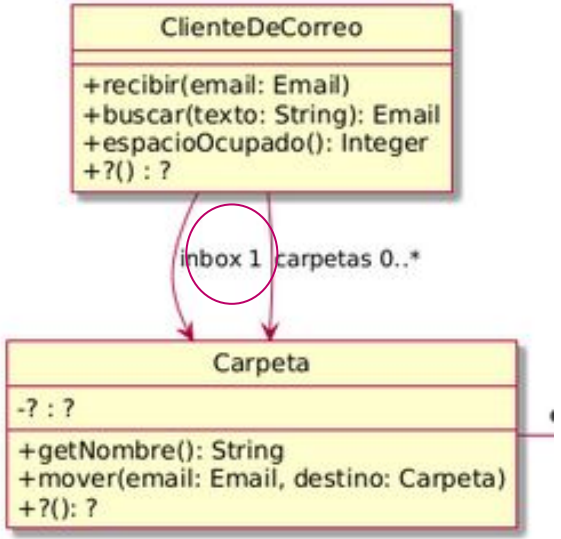


Diagrama de Clases - Asociación

- navegabilidad
- **multiplicidad**
- nombre de rol
- tipo:
  - simple
  - agregación
  - composición





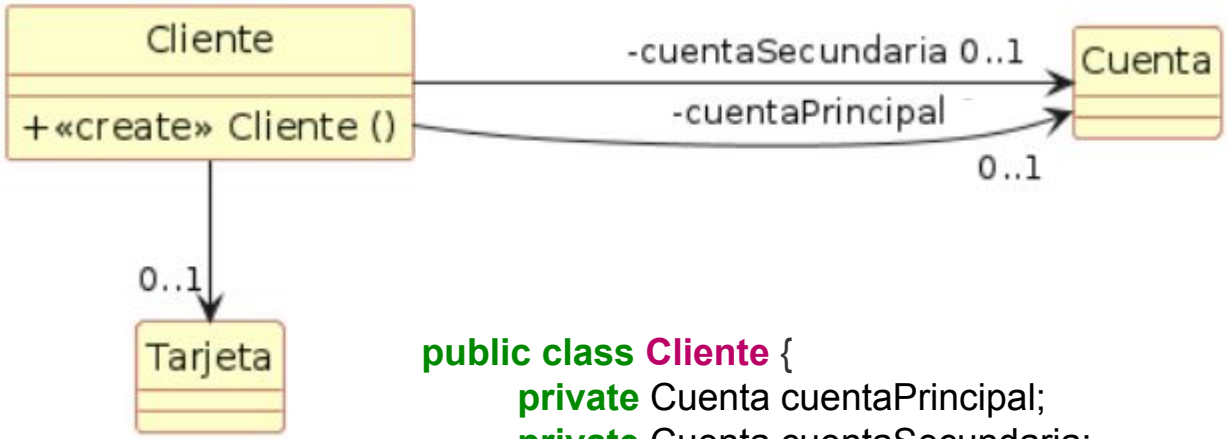

```
public ClienteDeCorreo() {

this.inbox = new Carpeta(...)

}
```

Diagrama de Clases - Asociación

- navegabilidad
- multiplicidad
- **nombre de rol**
- tipo:
  - simple
  - agregación
  - composición

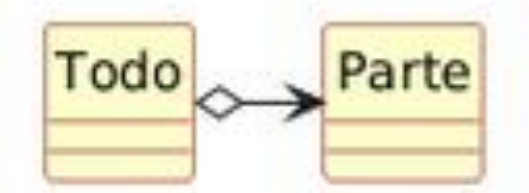


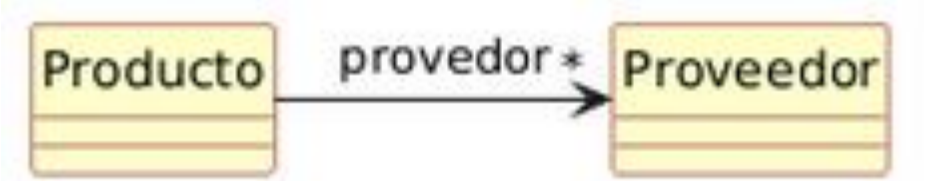

```
public class Cliente {
        private Cuenta cuentaPrincipal;
        private Cuenta cuentaSecundaria;
        private Tarjeta tarjeta;

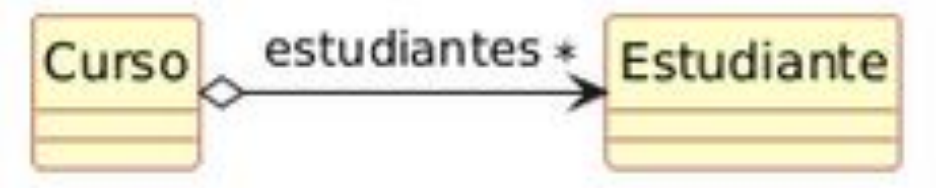
 }
```

Diagrama de Clases - Asociación

- navegabilidad
- multiplicidad
- nombre de rol
- **tipo:**
  - **simple**
  - **agregación**
  - **composición**







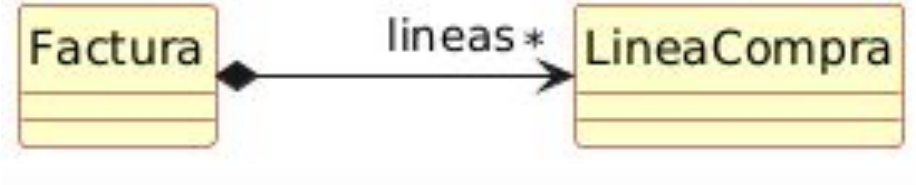


Diagrama de Clases - Interfaces



Una interfaz define un conjunto de operaciones que una clase o componente debe implementar.

Actúa como un contrato que establece qué métodos deben ser implementados

En UML, las interfaces se representan mediante un rectángulo con el nombre de la interfaz precedido por el estereotipo `<<interface>>`.

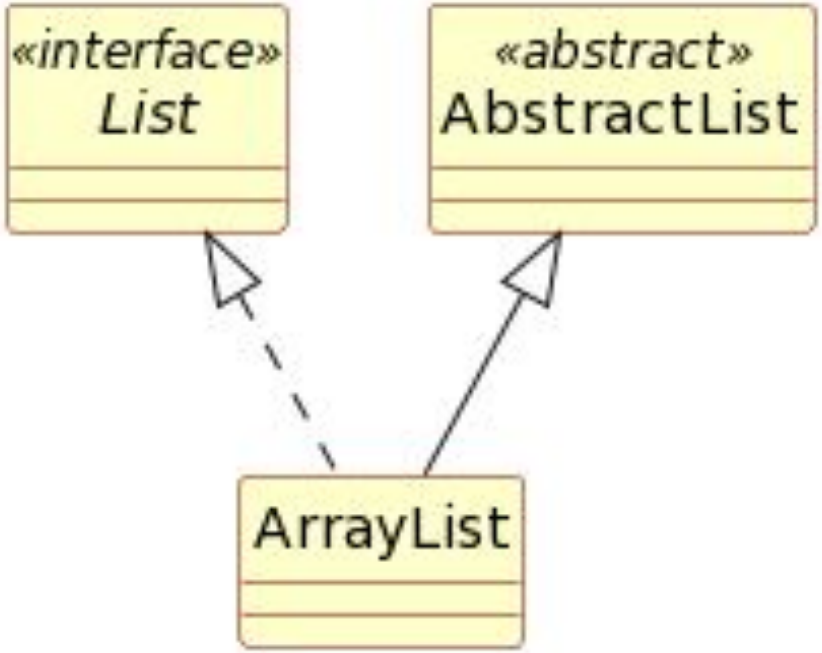
Dentro del rectángulo, se listan las operaciones o métodos que la interfaz define.

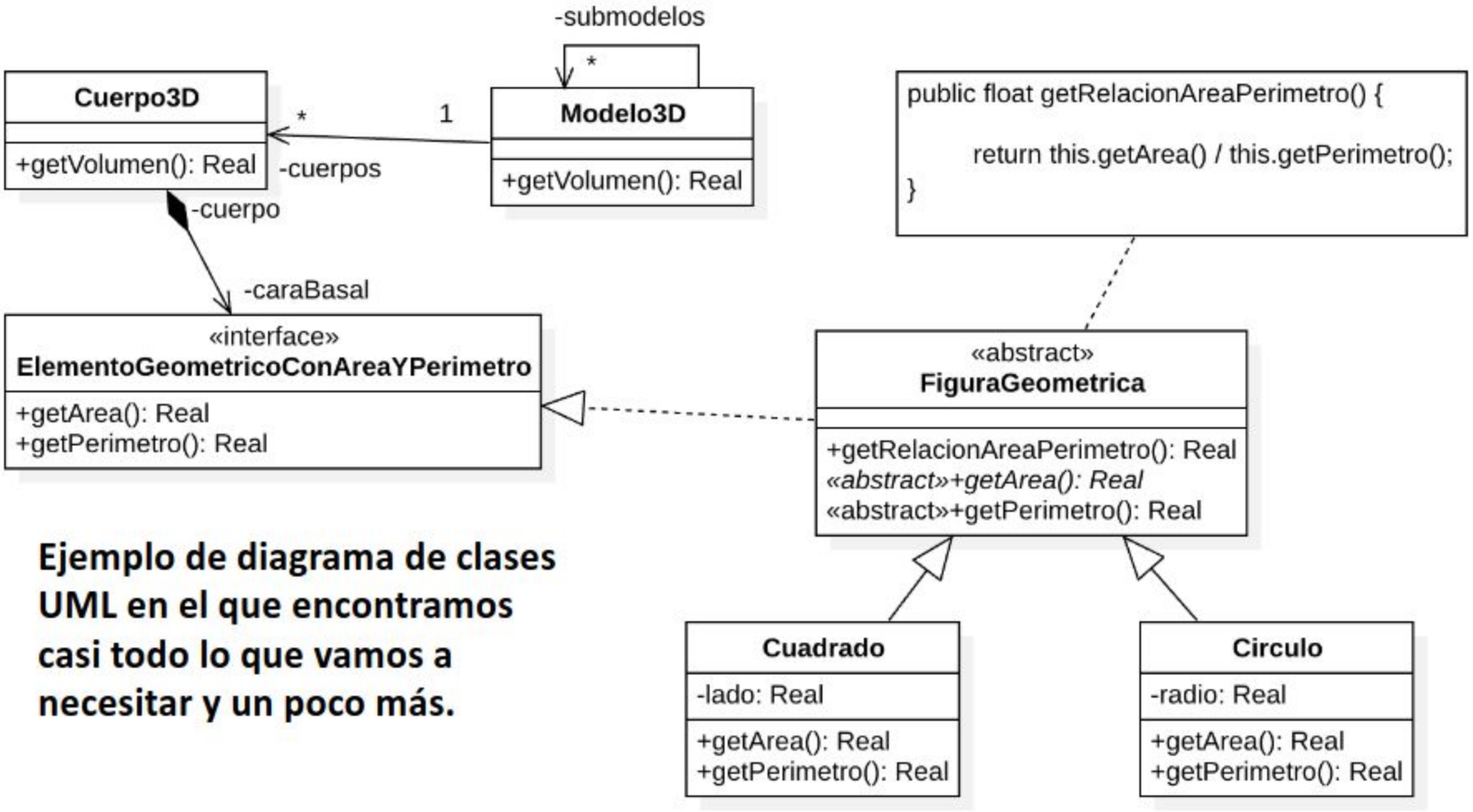
La implementación de los métodos definidos en la interfaz es responsabilidad de las clases que la implementan.

Diagrama de Clases - Interfaces y herencia

Una clase concreta implementa una interface. Esto significa que la clase proporciona una implementación concreta para todos los métodos (operaciones) definidos en esa interfaz.

Si una clase concreta extiende a una clase abstracta, significa que debe proveer las implementación de los métodos abstractos.





Ejemplo de diagrama de clases UML en el que encontramos casi todo lo que vamos a necesitar y un poco más.

Diagrama de secuencia

Un diagrama de secuencia es un tipo de diagrama de interacción porque describe cómo —y en qué orden— colabora un grupo de objetos.

- Muestra claramente cómo interactúan distintos objetos en un sistema a lo largo del tiempo.

En un diagrama de secuencia

- los objetos se representan en la parte superior del diagrama
- el tiempo avanza de arriba hacia abajo.
-

Diagrama de secuencia

Diagrama de secuencia del mensaje cargarPelicula de CinefilOOs

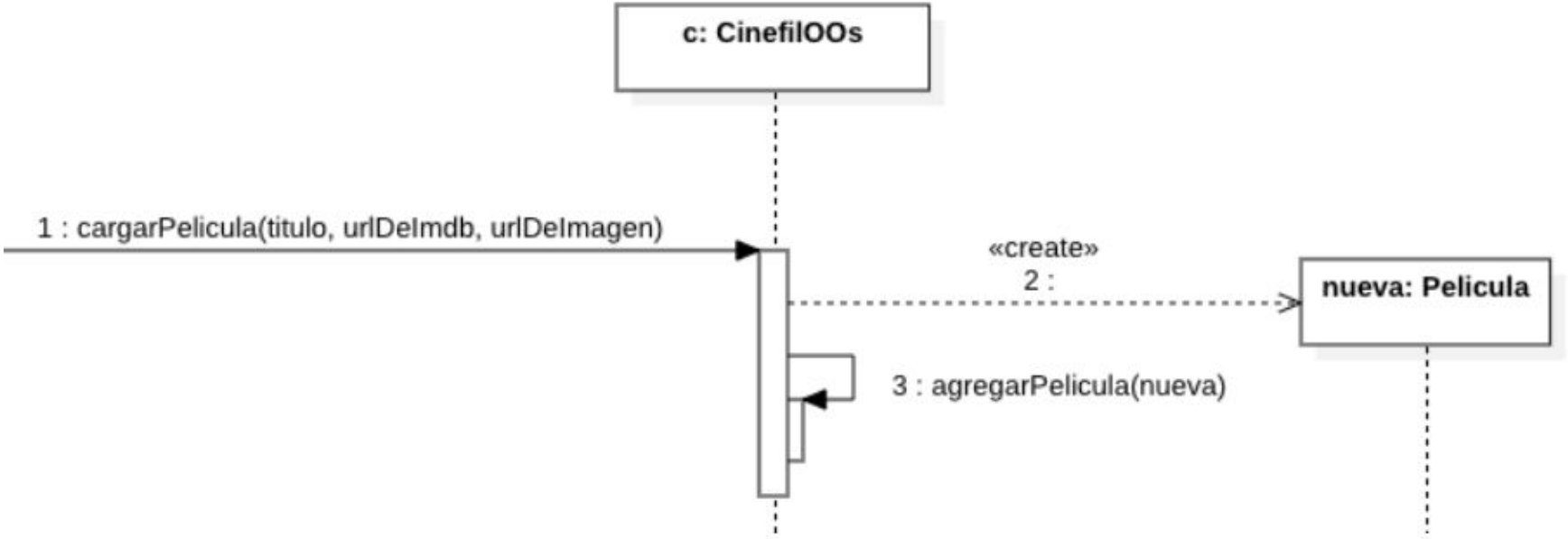


Diagrama de secuencia

- Las flechas horizontales muestran las interacciones entre los objetos, indicando quié
  envía un mensaje a quién y en qué orden.

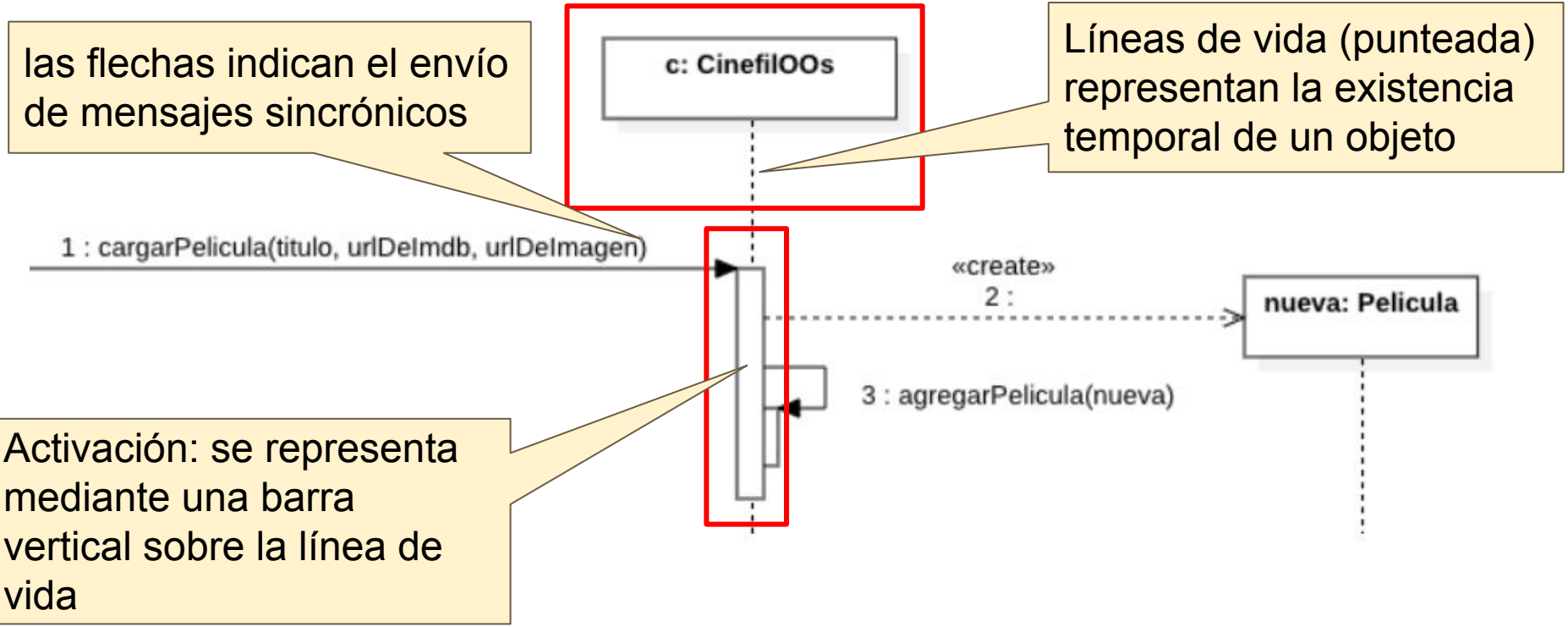


Diagrama de secuencia

- Las flechas horizontales muestran las interacciones entre los objetos, indicando quién envía un mensaje a quién y en qué orden.

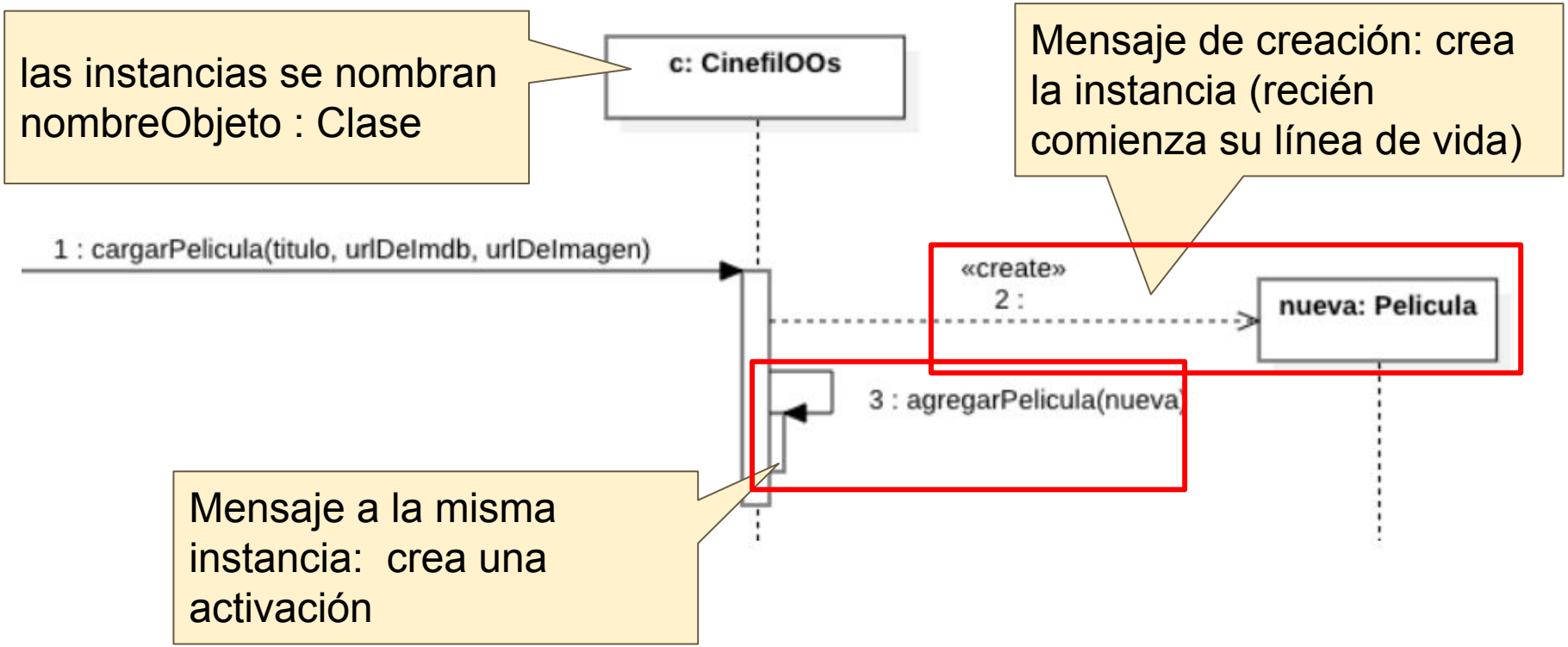


Diagrama de secuencia

Los corchetes indican que es opcional

Sintaxis del mensaje:

```
[valor de retorno := ] nombre del mensaje (parámetros)
```

Ejemplos:

```
agregarPelicula (nueva)

hayMonto:= hayMontoSuficienteParaExtraer(monto)
```

Diagrama de secuencia - CombinedFragment

Un fragmento combinado es un elemento que se utiliza para representar la lógica de control y las estructuras condicionales en una secuencia de interacción entre objetos. A través de ellos se pueden especificar bloques repetitivos, opcionales y alternativos, entre otros.

Fragmentos más utilizados:

- opt
- alt
- loop

Diagrama de secuencia - CombinedFragment

- **opt (Opcional)**: Representa una parte de la secuencia de interacción que puede o no ejecutarse, dependiendo de una condición booleana. Si la condición es verdadera, se ejecuta la parte opcional; de lo contrario, se omite.

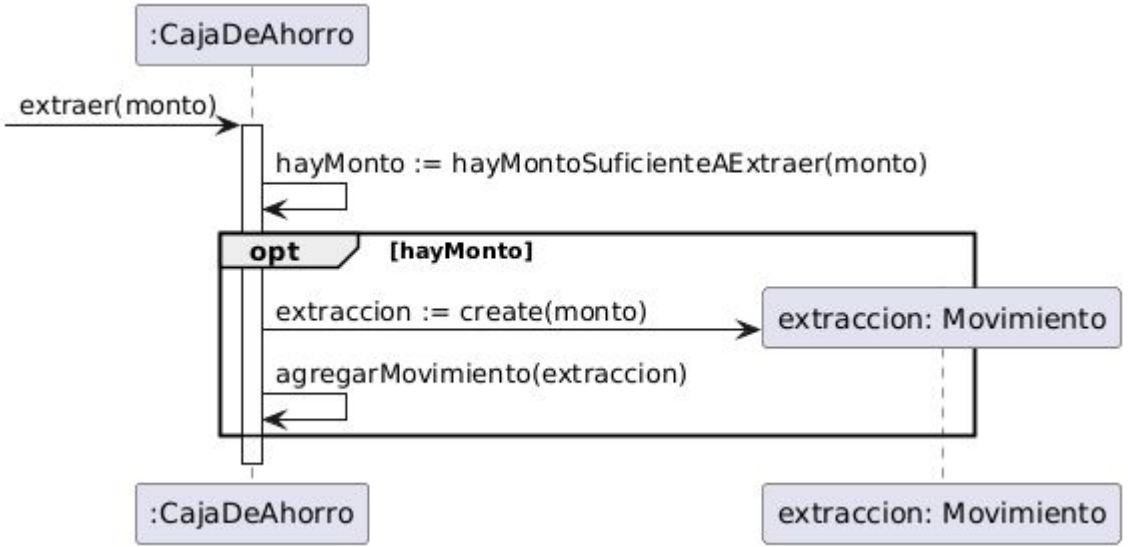


Diagrama de secuencia - CombinedFragment

- **alt (Alternativa)**: Este tipo de fragmento se utiliza para modelar una elección entre diferentes opciones de interacción. En cada opción se evalúa una condición booleana para determinar cuál de las opciones se ejecutará.

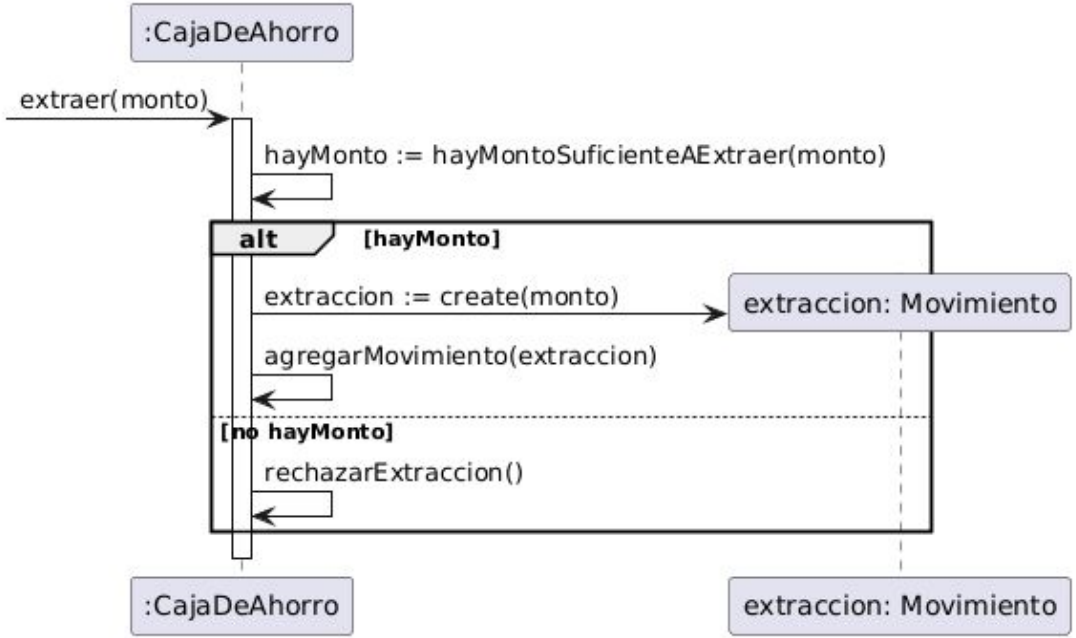
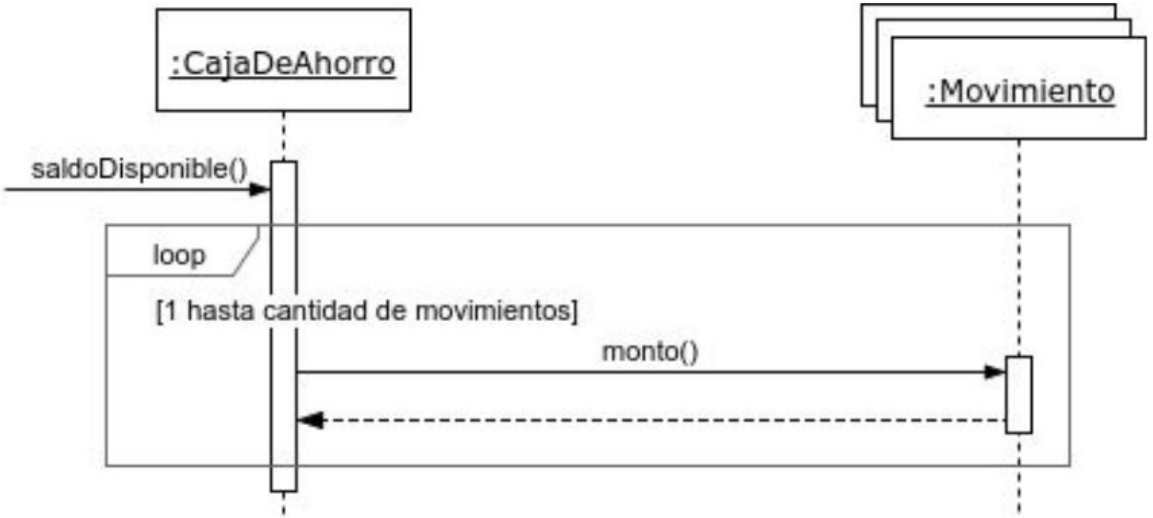


Diagrama de secuencia - CombinedFragment

loop (Bucle): Se utiliza para modelar repeticiones de una secuencia de interacción. Puede especificar el número de repeticiones o utilizar una condición para controlar la terminación del bucle.



¿ Preguntas ?